

CS420 Assignment 1

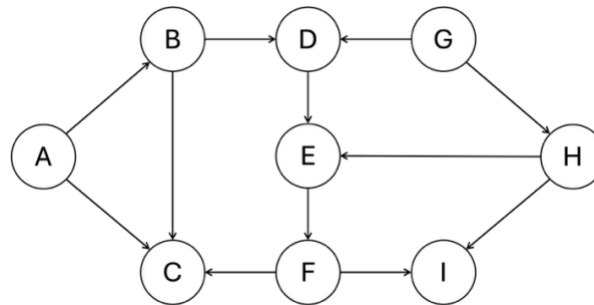
SOLUTION

Instructions: Your submission for this assignment should have two parts: **a PDF document and code files**, in a single zipped folder.

1. Have a **single PDF document** that contains your solutions to all questions (include detailed steps, results, and/or theoretical justifications as required).
2. If coding is required, include your relevant code snippets for each corresponding question in your solution PDF (e.g., as screenshots).
3. Upload the actual code files that you used to solve the questions. Make sure your code is clearly organized by question and runs correctly.
4. Combine your solution PDF and code files into a single zip file and upload it to eLearn.
5. The solution must be typeset using a professional software (Word, LaTeX, etc).
Handwritten solutions/drawings are NOT allowed and will not be graded.

Question 1 [10 points]:

Consider the following Bayesian network:



1) Mark **True/False** for the following conditional independence statements implied by the BN. **Justify your answers clearly.** For example, show the active/blocked trails as necessary and the reason for them to be active or blocked (e.g., use rules such as cascade, common cause or v-structure as applicable on a particular node in the trail). **[Each 1.5 points]** (no coding is required for this question)

a) $B \perp G$

True

All trails are blocked (**11** trails):

$B \rightarrow D \leftarrow G$ is blocked by D (v-structure, blocked since neither D nor any of its descendants is observed)

$B \rightarrow D \rightarrow E \leftarrow H \leftarrow G$ is blocked by E (v-structure, blocked since neither E nor any of its descendants is observed)

$B \rightarrow D \rightarrow E \rightarrow F \rightarrow I \leftarrow H \leftarrow G$ is blocked by I (v-structure, blocked since neither I nor any of its descendants is observed)

4 trails starting with $B \leftarrow A \rightarrow C \leftarrow F \dots$ are blocked by C (v-structure, blocked since neither C nor any of its descendants is observed)

4 trails starting with $B \rightarrow C \leftarrow F \dots$ are blocked by C (v-structure, blocked since neither C nor any of its descendants is observed)

b) $C \perp H \mid I$

False

Any **one** active trail is sufficient:

$C \leftarrow F \rightarrow I \leftarrow H$ is active: F (common cause, active as F is not observed), I (v-structure, active as I is observed)

c) $A \perp H \mid \{B, F\}$

True

All trails are blocked (**12** trails):

3 trails starting with $A \rightarrow B \rightarrow D \dots$ are blocked by B (cascade, blocked as B is observed)

3 trails starting with $A \rightarrow B \rightarrow C \dots$ are blocked by B (cascade, blocked as B is observed)

3 trails starting with $A \rightarrow C \leftarrow F \dots$ are blocked by C (v-structure, blocked since neither C nor any of its descendants is observed)

3 trails starting with $A \rightarrow C \leftarrow B \dots$ are blocked by C (v-structure, blocked since neither C nor any of its descendants is observed)

d) $A \perp \{G, I\} \mid \{B, C\}$

False

Any one active trail is sufficient:

$A \rightarrow C \leftarrow F \rightarrow I$ is active: C (v-structure, active as C is observed), F (common cause, active as F is not observed)

2) Construct the above Bayesian network using **pgmpy**, and verify statements (1a) – (1d) using d-separation. For each statement, output whether it is True or False according to d-separation. The skeleton code is provided in **Q1.ipynb**. [4 points]

(Also remember to attach the screenshots of your code snippets in your solution pdf)

Create the Bayesian network model

```
# Create the BN model which contains edges of the DAG graph
# Note that CPDs are not required; D-separation depends only on the graph structure

## write your code here [2 points]
model = DiscreteBayesianNetwork([('A', 'B'), ('A', 'C'), ('B', 'C'), ('B', 'D'),
                                  ('D', 'E'), ('E', 'F'), ('F', 'C'), ('F', 'I'),
                                  ('G', 'D'), ('G', 'H'), ('H', 'E'), ('H', 'I'))])
```

Verify the d-separation results for (1a) – (1d)

```
# Use the d-separation method to check the conditional independence statements in (1a) – (1d)

## write your code here [2 points]
# Check (1a) whether B and G are d-separated
print('B and G are d-separated:', model.is_dseparated(['B'], ['G']))

# Check (1b) whether C and H are d-separated given I
print('C and H are d-separated given {I}:', model.is_dseparated(['C'], ['H'], observed=['I']))

# Check (1c) whether A and H are d-separated given {B, F}
print('A and H are d-separated given {B,F}:', model.is_dseparated(['A'], ['H'], observed=['B', 'F']))

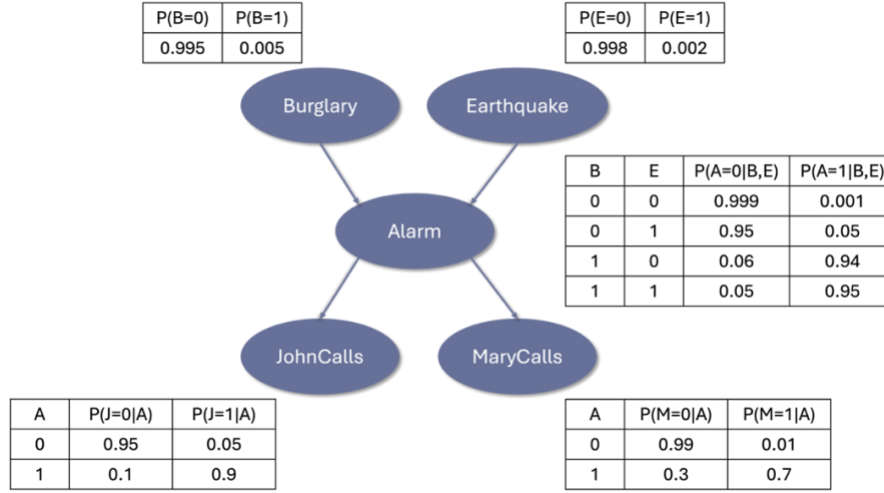
# Check (1d) whether A and {G, I} are d-separated given {B, C}
print('A and {G, I} are d-separated given {B,C}:', model.is_dseparated(['A'], ['G', 'I'], observed=['B', 'C']))

... B and G are d-separated: True
    C and H are d-separated given {I}: False
    A and H are d-separated given {B,F}: True
    A and {G, I} are d-separated given {B,C}: False
```

Question 2 [12 points]:

You have a burglar alarm installed in your home. The alarm is fairly reliable at detecting burglaries, but it may also be triggered by minor earthquakes. You have two neighbors, John and Mary, who have agreed to call you if they hear the alarm. John almost always calls when he hears the alarm, while Mary is more likely to miss the alarm.

The Bayesian network and its conditional probability distributions (CPDs) are shown in the figure. We use the following notation: **B**: Burglary, **E**: Earthquake, **A**: Alarm, **J**: JohnCalls, **M**: MaryCalls. Each variable takes values in $\{0, 1\}$.



1) Compute the marginal inference $P(J)$ using **Variable Elimination**. You need to compute and show **all intermediate factors/tables** created during the elimination process (i.e., explicitly compute the entries for all intermediate tables). **[6 points]**

Variable Elimination (any elimination order is acceptable)

Elimination order: M, E, B, A .

$$\begin{aligned}
 P(J) &= \sum_{B,E,A,M} P(B,E,A,J,M) \\
 &= \sum_A \sum_B \sum_E \sum_M P(B)P(E)P(A|B,E)P(J|A)P(M|A) && // \text{BN Factorization} \\
 &= \sum_A \sum_B \sum_E P(B)P(E)P(A|B,E)P(J|A) \sum_M P(M|A) && // \text{Collect factors involving } M \\
 &= \sum_A \sum_B \sum_E P(B)P(E)P(A|B,E)P(J|A)\tau_1(A) && // \text{Sum out to eliminate } M \text{ (may replace } \tau_1(A) = 1) \\
 &= \sum_A \sum_B P(B)P(J|A)\tau_1(A) \sum_E P(E)P(A|B,E) && // \text{Collect factors involving } E \\
 &= \sum_A \sum_B P(B)P(J|A)\tau_1(A)\tau_2(A,B) && // \text{Sum out to eliminate } E \\
 &= \sum_A P(J|A)\tau_1(A) \sum_B P(B)\tau_2(A,B) && // \text{Collect factors involving } B \\
 &= \sum_A P(J|A)\tau_1(A)\tau_3(A) && // \text{Sum out to eliminate } B \\
 &= \sum_A P(J|A)\tau_1(A)\tau_3(A) && // \text{Collect factors involving } A \\
 &= \tau_4(J) && // \text{Sum out to eliminate } A
 \end{aligned}$$

Now we compute all intermediate factors/tables $\tau_1(A)$, $\tau_2(A, B)$, $\tau_3(A)$, $\tau_4(J)$, where the result $P(J) = \tau_4(J)$.

For $\tau_1(A) = \sum_M P(M|A)$, note that the conditional probabilities $P(M|A)$ form a probability distribution over M , it follows from the normalization property that $\tau_1(A) = 1$, i.e., the table $\tau_1(A = 0) = 1$ and $\tau_1(A = 1) = 1$ for $A \in \{0, 1\}$.

For $\tau_2(A, B) = \sum_E P(E)P(A|B, E) = P(E = 0)P(A|B, E = 0) + P(E = 1)P(A|B, E = 1)$.

A	B	Calculation	$\tau_2(A, B)$
0	0	$0.998 \times 0.999 + 0.002 \times 0.95$	0.998902
0	1	$0.998 \times 0.06 + 0.002 \times 0.05$	0.05998
1	0	$0.998 \times 0.001 + 0.002 \times 0.05$	0.001098
1	1	$0.998 \times 0.94 + 0.002 \times 0.95$	0.94002

For $\tau_3(A) = \sum_B P(B)\tau_2(A, B) = P(B = 0)\tau_2(A, B = 0) + P(B = 1)\tau_2(A, B = 1)$.

A	Calculation	$\tau_3(A)$
0	$0.995 \times 0.998902 + 0.005 \times 0.05998$	0.99420739
1	$0.995 \times 0.001098 + 0.005 \times 0.94002$	0.00579261

The final marginal inference $P(J) = \tau_4(J) = \sum_A P(J|A)\tau_1(A)\tau_3(A) = P(J|A = 0)\tau_3(A = 0) + P(J|A = 1)\tau_3(A = 1)$, note $\tau_1(A) = 1$.

J	Calculation	$P(J) = \tau_4(J)$
0	$0.95 \times 0.99420739 + 0.1 \times 0.00579261$	0.9450762815
1	$0.05 \times 0.99420739 + 0.9 \times 0.00579261$	0.0549237185

2) Compute the posterior inference $P(B | M = 1)$ using **Variable Elimination**. You need to compute and show **all intermediate factors/tables** created during the elimination process (i.e., explicitly compute the entries for all intermediate tables). Then **normalize** the result to obtain the final posterior distribution. **[6 points]** (Hint: $P(B | M = 1) = \frac{P(B, M = 1)}{P(M = 1)} = \frac{P(B, M = 1)}{\sum_B P(B, M = 1)}$)

Variable Elimination (any elimination order is acceptable)

Elimination order: J, E, A .

$$\begin{aligned}
P(B, M = 1) &= \sum_{E, A, J} P(B, E, A, J, M = 1) \\
&= \sum_A \sum_E \sum_J P(B)P(E)P(A|B, E)P(J|A)P(M = 1|A) && // \text{BN Factorization} \\
&= \sum_A \sum_E P(B)P(E)P(A|B, E)P(M = 1|A) \sum_J P(J|A) && // \text{Collect factors involving } J \\
&= \sum_A \sum_E P(B)P(E)P(A|B, E)P(M = 1|A)\tau_1(A) && // \text{Sum out to eliminate } J \text{ (may replace } \tau_1(A) = 1) \\
&= \sum_A P(B)P(M = 1|A)\tau_1(A) \sum_E P(E)P(A|B, E) && // \text{Collect factors involving } E \\
&= \sum_A P(B)P(M = 1|A)\tau_1(A)\tau_2(A, B) && // \text{Sum out to eliminate } E \\
&= P(B) \sum_A P(M = 1|A)\tau_1(A)\tau_2(A, B) && // \text{Collect factors involving } A \\
&= P(B)\tau_3(M = 1, B) && // \text{Sum out to eliminate } A
\end{aligned}$$

Now we compute all intermediate factors/tables $\tau_1(A)$, $\tau_2(A, B)$, $\tau_3(M = 1, B)$, where the result $P(B, M = 1) = P(B)\tau_3(M = 1, B)$.

For $\tau_1(A) = \sum_J P(J|A)$, note that the conditional probabilities $P(J|A)$ form a probability distribution over J , it follows from the normalization property that $\tau_1(A) = 1$, i.e., the table $\tau_1(A = 0) = 1$ and $\tau_1(A = 1) = 1$ for $A \in \{0, 1\}$.

For $\tau_2(A, B) = \sum_E P(E)P(A|B, E) = P(E = 0)P(A|B, E = 0) + P(E = 1)P(A|B, E = 1)$.

A	B	Calculation	$\tau_2(A, B)$
0	0	$0.998 \times 0.999 + 0.002 \times 0.95$	0.998902
0	1	$0.998 \times 0.06 + 0.002 \times 0.05$	0.05998
1	0	$0.998 \times 0.001 + 0.002 \times 0.05$	0.001098
1	1	$0.998 \times 0.94 + 0.002 \times 0.95$	0.94002

For $\tau_3(M = 1, B) = \sum_A P(M = 1|A)\tau_1(A)\tau_2(A, B) = P(M = 1|A = 0)\tau_2(A = 0, B) + P(M = 1|A = 1)\tau_2(A = 1, B)$, note $\tau_1(A) = 1$.

B	Calculation	$\tau_3(M = 1, B)$
0	$0.01 \times 0.998902 + 0.7 \times 0.001098$	0.01075762
1	$0.01 \times 0.05998 + 0.7 \times 0.94002$	0.6586138

For $P(B, M = 1) = P(B)\tau_3(M = 1, B)$

B	Calculation	$P(B, M = 1)$
0	0.995×0.01075762	0.0107038319
1	0.005×0.6586138	0.003293069

The final posterior inference $P(B|M = 1)$ can be obtained via normalization, $P(B|M = 1) = \frac{P(B, M=1)}{\sum_B P(B, M=1)} = \frac{P(B, M=1)}{\sum_B P(B, M=1)}$.

B	Calculation	$P(B M = 1)$
0	$\frac{0.0107038319}{0.0107038319+0.003293069}$	0.764728705
1	$\frac{0.003293069}{0.0107038319+0.003293069}$	0.235271295

Question 3 [8 points]:

In this question, you will construct the Bayesian network from **Question 2** using the given graph structure and CPDs. The skeleton code is provided in **Q3.ipynb**.

Tasks:

- Build the Bayesian network model using **pgmpy** for the DAG graph given in **Question 2**. [1 point]

Creating the Bayesian network model

```

▶ # (3a) Create the BN model for the DAG graph given in Question 2

# write your code here [1 point]
model = DiscreteBayesianNetwork([('Burglary', 'Alarm'),
                                  ('Earthquake', 'Alarm'),
                                  ('Alarm', 'JohnCalls'),
                                  ('Alarm', 'MaryCalls')])

```

- Add the CPDs (given in **Question 2**) for all variables to the model. [3 points]

Add the CPDs

```

▶ # (3b) Add all CPDs (given in Question 2) to the model

# write your code here [3 points]
cpd_burglary = TabularCPD(variable='Burglary', variable_card=2,
                           values=[[0.995], [0.005]])

cpd_earthquake = TabularCPD(variable='Earthquake', variable_card=2,
                              values=[[0.998], [0.002]])

cpd_alarm = TabularCPD(variable='Alarm', variable_card=2,
                        values=[[0.999, 0.95, 0.06, 0.05], # P(A=0|B,E)
                                [0.001, 0.05, 0.94, 0.95]], # P(A=1|B,E)
                        evidence=['Burglary', 'Earthquake'], evidence_card=[2, 2])

cpd_john = TabularCPD(variable='JohnCalls', variable_card=2,
                       values=[[0.95, 0.1], # P(J=0|A)
                                [0.05, 0.9]], # P(J=1|A)
                       evidence=['Alarm'], evidence_card=[2])

cpd_mary = TabularCPD(variable='MaryCalls', variable_card=2,
                       values=[[0.99, 0.3], # P(M=0|A)
                                [0.01, 0.7]], # P(M=1|A)
                       evidence=['Alarm'], evidence_card=[2])

model.add_cpds(cpd_burglary, cpd_earthquake, cpd_alarm, cpd_john, cpd_mary)

```

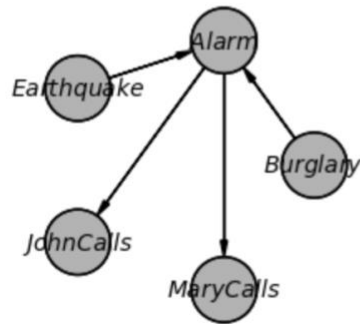
- c) Perform a model check and plot your Bayesian network graph. [2 points]

Model check and plot the BN

```
# (3c) Perform a model check and plot your Bayesian network graph

# write your code here [2 points]
print("Model check:", model.check_model())
model_daft = model.to_daft()
model_daft.render()
plt.show()
```

... Model check: True



- d) Perform inference using **VariableElimination** to verify your final results for **Question 2**, i.e., $P(J)$ and $P(B \mid M = 1)$. [2 points]

```
# (3d) Perform inference using VariableElimination to verify your final results for Question 2

# write your code here [2 points]
infer = VariableElimination(model)

p_J = infer.query(variables=['JohnCalls'])
print("Marginal Inference P(J):\n", p_J)

p_B_given_M1 = infer.query(variables=['Burglary'], evidence={'MaryCalls': 1})
print("\nPosterior Inference P(B|M=1):\n", p_B_given_M1)
```

... Marginal Inference P(J):

JohnCalls	phi(JohnCalls)
JohnCalls(0)	0.9451
JohnCalls(1)	0.0549

Posterior Inference P(B|M=1):

Burglary	phi(Burglary)
Burglary(0)	0.7647
Burglary(1)	0.2353

Question 4 [15 points]:

In this question, you will build a **Convolutional Neural Network (CNN)** classifier for the **CIFAR-10** dataset. Dataset information: <https://www.cs.toronto.edu/~kriz/cifar.html>.

Your network must include the following components:

- An input layer
- **At least one** convolutional layer
- **At least one** max-pooling layer
- **At least one** dense (fully connected) layer
- An output layer with softmax activation to produce probabilities over the 10 classes

The skeleton code is provided in **Q4.ipynb**.

Tasks:

- a) Load the CIFAR-10 dataset and perform normalization preprocessing. [1 point]

Any correct code is acceptable. An example is provided below.

```
# (4a) Load the CIFAR-10 dataset https://keras.io/datasets/
# Normalize pixel values from [0, 255] to [0, 1] (e.g., divide by 255)

## write your code here [1 point]
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.cifar10.load_data()

train_images = train_images / 255.0
test_images = test_images / 255.0

... Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 14s 0us/step
```

- b) Build a CNN that includes **at least one** convolutional layer, **at least one** max-pooling layer, and **at least one** dense layer. [3 points]

Any correct code is acceptable. An example is provided below.

```
# (4b) Build your Convolutional Neural Network (CNN) that includes
# at least one convolutional layer
# at least one max-pooling layer
# at least one dense layer

## write your code here [3 points]
model = tf.keras.Sequential()
model.add(layers.Input(shape=(32, 32, 3)))

model.add(layers.Conv2D(64, kernel_size=(3, 3), strides=(1, 1), activation="relu"))
model.add(layers.Conv2D(64, kernel_size=(5, 5), strides=(1, 1), padding="same", activation="relu"))
model.add(layers.MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))
model.add(layers.Conv2D(128, kernel_size=(3, 3), strides=(1, 1), padding="same", activation="relu"))
model.add(layers.Conv2D(128, kernel_size=(3, 3), strides=(1, 1), padding="same", activation="relu"))
model.add(layers.Flatten())
model.add(layers.Dense(128, activation="relu"))
model.add(layers.Dense(10, activation="softmax"))
```

c) For each layer in your network, compute:

- The output shape
- The number of trainable parameters

Show your **calculation steps** for obtaining these values. [3 points]

The calculation steps and results should correspond to the CNN built in part b).

(4c) Compute the output shape and number of parameters required for each layer.

Show your **calculation steps** for obtaining these values. [3 points]

Input image shape: 32x32x3

($H1 = W1 = (32-3)/1+1 = 30$, $D1 = \text{\#filters} = 64$)

Layer 1 output shape: 30x30x64; parameters: $64 \times (3 \times 3 \times 3 + 1) = 1792$

(same padding: $H2 = H1$, $W2 = W1$, $D2 = \text{\#filters} = 64$)

Layer 2 output shape: 30x30x64; #parameters: $64 \times (5 \times 5 \times 64 + 1) = 102464$

($H3 = W3 = (30-2)/2+1 = 15$, $D3 = D2 = 64$)

Layer 3 output shape: 15x15x64; #parameters: 0

(same padding: $H4 = H3$, $W4 = W3$, $D4 = \text{\#filters} = 128$)

Layer 4 output shape: 15x15x128; #parameters: $128 \times (3 \times 3 \times 64 + 1) = 73856$

(same padding: $H5 = H4$, $W5 = W4$, $D5 = \text{\#filters} = 128$)

Layer 5 output shape: 15x15x128; Layer 5 #parameters: $128 \times (3 \times 3 \times 128 + 1) = 147584$

($15 \times 15 \times 128 = 28800$)

Layer 6 output shape: 28800; Layer 6 #parameters: 0

(output shape is the #neurons = 128)

Layer 7 output shape: 128; #parameters: $128 \times (28800 + 1) = 3686528$

(output shape is the #neurons = 10)

Layer 8 output shape: 10; #parameters: $10 \times (128 + 1) = 1290$

▶ # (4c) Verify your results via `model.summary()`
`model.summary()`

... Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 30, 30, 64)	1,792
conv2d_5 (Conv2D)	(None, 30, 30, 64)	102,464
max_pooling2d_1 (MaxPooling2D)	(None, 15, 15, 64)	0
conv2d_6 (Conv2D)	(None, 15, 15, 128)	73,856
conv2d_7 (Conv2D)	(None, 15, 15, 128)	147,584
flatten_1 (Flatten)	(None, 28800)	0
dense_2 (Dense)	(None, 128)	3,686,528
dense_3 (Dense)	(None, 10)	1,290

Total params: 4,013,514 (15.31 MB)

Trainable params: 4,013,514 (15.31 MB)

Non-trainable params: 0 (0.00 B)

- d) Specify your training configuration, including the **loss function, optimizer/optimization method, learning rate, batch size, and number of epochs**. [1 point]

Any correct code is acceptable. An example is provided below.

```
# (4d) Specify your training configuration, including
# loss function, optimizer, learning rate, batch size, number of epochs

## write your code here [1 point]
loss = tf.keras.losses.SparseCategoricalCrossentropy()
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
batch_size = 64
epochs = 20

model.compile(loss=loss, optimizer=optimizer, metrics=['accuracy'])
```

- e) Plot the **training loss** as a function of epochs (i.e., plot the training loss at each epoch). [3 points]

Any correct code is acceptable. An example is provided below.

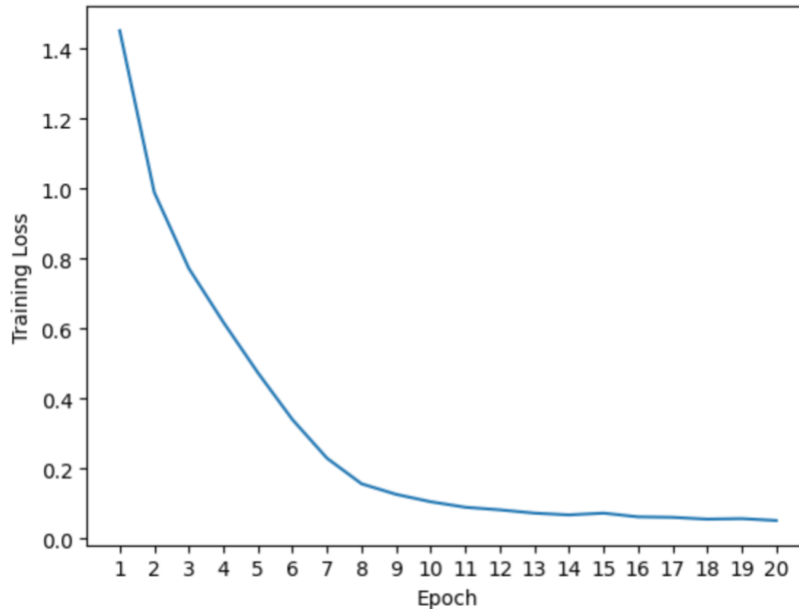
```
# (4e) Plot the training loss as a function of epochs

## write your code here for training and plotting [3 points]
history = model.fit(train_images, train_labels, batch_size=batch_size, epochs=epochs)

plt.plot(range(1, epochs + 1), history.history["loss"])
plt.xlabel("Epoch")
plt.ylabel("Training Loss")
plt.xticks(range(1, epochs + 1))
plt.show()
```

```
Epoch 1/20
782/782 ————— 18s 19ms/step - accuracy: 0.4717 - loss: 1.4509
Epoch 2/20
782/782 ————— 14s 18ms/step - accuracy: 0.6514 - loss: 0.9890
Epoch 3/20
782/782 ————— 14s 18ms/step - accuracy: 0.7286 - loss: 0.7721
Epoch 4/20
782/782 ————— 20s 17ms/step - accuracy: 0.7826 - loss: 0.6182
Epoch 5/20
782/782 ————— 14s 17ms/step - accuracy: 0.8340 - loss: 0.4739
Epoch 6/20
782/782 ————— 14s 17ms/step - accuracy: 0.8800 - loss: 0.3401
Epoch 7/20
782/782 ————— 14s 18ms/step - accuracy: 0.9180 - loss: 0.2299
Epoch 8/20
782/782 ————— 14s 18ms/step - accuracy: 0.9449 - loss: 0.1569
Epoch 9/20
782/782 ————— 14s 18ms/step - accuracy: 0.9551 - loss: 0.1268
Epoch 10/20
782/782 ————— 14s 18ms/step - accuracy: 0.9633 - loss: 0.1057
Epoch 11/20
782/782 ————— 14s 17ms/step - accuracy: 0.9698 - loss: 0.0900
Epoch 12/20
782/782 ————— 14s 17ms/step - accuracy: 0.9722 - loss: 0.0826
Epoch 13/20
782/782 ————— 14s 17ms/step - accuracy: 0.9745 - loss: 0.0733
Epoch 14/20
```

Epoch 14/20
782/782 ————— **14s** 17ms/step — accuracy: 0.9772 — loss: 0.0681
 Epoch 15/20
782/782 ————— **14s** 17ms/step — accuracy: 0.9758 — loss: 0.0732
 Epoch 16/20
782/782 ————— **14s** 17ms/step — accuracy: 0.9791 — loss: 0.0626
 Epoch 17/20
782/782 ————— **14s** 17ms/step — accuracy: 0.9795 — loss: 0.0612
 Epoch 18/20
782/782 ————— **14s** 17ms/step — accuracy: 0.9820 — loss: 0.0561
 Epoch 19/20
782/782 ————— **14s** 17ms/step — accuracy: 0.9806 — loss: 0.0575
 Epoch 20/20
782/782 ————— **14s** 17ms/step — accuracy: 0.9823 — loss: 0.0521



- f) Report your **test accuracy**. The test accuracy should be **at least 70%**. You may improve the accuracy by modifying your network architecture (e.g., adding more layers/neurons) or adjusting the optimizer, learning rate, etc. **[4 points]**

4 pts if test accuracy $\geq 70\%$

3 pts if test accuracy $\geq 65\%$

2 pts if test accuracy $\geq 60\%$

1 pt if test accuracy $< 60\%$

0 pt if test accuracy is not reported

```
# (4f) Report your test accuracy. The test accuracy should be at least 70%

## write your code here [4 points]
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f'Test accuracy is {test_acc * 100:.2f}%')
```

... **313/313** ————— **3s** 6ms/step — accuracy: 0.7055 — loss: 2.2882
 Test accuracy is 70.55%